```
    self.image = self.photos.get_next_photo()
    if self.image == None:
        self.root.quit()
    else:
        self.display_image(self.image)
```

The above method gets the next image from the *photos* object and calls the *display image* method. If there are no more images, the application quits.

```
def save_and_next(self):
    self.photos.save()
    self.next_image()
```

The above method calls the *save* method of the *photos* object and then continues to display the next image.

```
def apply_caption(self, event):
    text = self.caption.get()
    self.image = self.photos.photo_with_caption(text)
    self.display_image(self.image)
```

The *apply_caption* method is called when the *Return* or *Enter* key is pressed after entering the caption text. The modified image is displayed.

The code to create and start the application is as follows:

```
my_photos = photos((800,600))  # convert images to 800x600
app = gui(my_photos)
```

Font selection

If the text is too small, you can load and select your own font. The following lines of code will allow you to use your own font and size:

```
import ImageFont
self.font = ImageFont.truetype('/usr/share/fonts/lohit-hindi/lohit_
hi.ttf',30)
draw.text((50,self.new_size[1] - 50), caption,font=self.font,fill='#00f')
```

Not surprisingly, you can now enter the caption in Hindi, and in blue colour.

Jazz up the transitions

You can use your little application as a simple viewer as well. Just keep skipping each photograph! That's justification enough to explore some more capabilities of the imaging module.

You might like to have a fading effect whenever the next photograph is displayed. You need to write a method that will generate a sequence of images, starting with the current image and ending up with the new one. In the photos class, add the following method:

```
def generate_photo_transition(self):
    try:
        old_image = self.image
        new_image = self.get_next_photo()
        for transition in range(10):
            self.transition_image = Image.blend(old_image, new_image,
0.1*(transition + 1))
            yield self.transition_image
    except Exception,e:
        yield new_image
```

The generator for the sequence of images is simple. Just use a *yield* statement to return an image. The *blend* function from the *image* module is used to create a new image, which is a linear combination of the two images—*(1 – r)*old + r*new*. The factor *r* is the third parameter.

If there isn't an old or new image, an exception will be raised. If no old image exists, the new image will be displayed with no transition effect. If no new image exists, a null value will be returned and the GUI will terminate.

The GUI program will need to iterate over the generator. The revised *next_image* method will be more complex:

```
def next_image(self):
    for self.image in self.photos.generate_photo_transition():
        if self.image == None:
            self.root.quit()
        else:
            self.display_image(self.image)
            self.foto.update_idletasks()
        time.sleep(.1)
```

The key difference is that you are now iterating over the generator of the transition images. Each intermediary image is displayed and the application sleeps for a while. However, by default, it will not be updated until the control reverts to the main loop. The method *update_idletasks* forces the image to be displayed.

Obviously, the Python imaging module can do a lot more than can be covered in one article. You can use it to convert between formats, apply filters, enhance images, apply geometric transformations, manipulate pixels, crop and paste regions, manipulate frames in animated images, etc. In short, you can use it to convert your collection of photographs into a memorable set of pictures that you would love to see over and over. Also, you will not bore your friends with an endless stream of random pictures, where the good ones are lost in the clutter. END

By: Dr. Anil Seth

The author is a consultant by profession and can be reached at seth.anil@gmail.com